Primer: Deep Maximum Causal Entropy IRL

econirl package documentation

1 Problem and Motivation

Linear MCE-IRL (Ziebart, 2010) assumes that the reward function takes the form $R(s, a) = \theta^\top \phi(s, a)$, where ϕ is a hand-designed feature vector and θ is an unknown weight vector recovered from data. This linear parameterization works well when the analyst knows the correct reward features in advance. When the true reward depends on higher-order interactions or nonlinear transformations of state variables, however, a linear model misspecifies the reward and produces suboptimal policies.

Wulfmeier et al. (2016) introduced Deep MCE-IRL, which replaces the linear reward with a neural network $R_\psi(s, a)$. The network learns both the reward function and the features that represent it directly from demonstration data. The MCE framework stays the same. The soft Bellman recursion stays the same. The occupancy measure computation stays the same. The only change is the reward parameterization and how the gradient propagates through it.

The motivation is environments where the reward depends on complex spatial or relational structure that is difficult to specify as hand-crafted features. The Objectworld environment (Wulfmeier et al., 2016) is the canonical test case. The reward there depends nonlinearly on distances to multiple colored objects, so no linear combination of continuous distance features can recover it exactly. A neural network can in principle learn this nonlinear mapping, though doing so on small discrete grids introduces its own challenges that we discuss in the Results section.

2 Model and Notation

The agent solves the same maximum causal entropy problem as in linear MCE-IRL

$$\max_{\pi} \; \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \big(R_\psi(s_t, a_t) + \sigma \mathcal{H}(\pi(\cdot|s_t)) \big) \right] \tag{1}$$

subject to the feature matching constraint. The only difference is that $R_\psi(s, a)$ is a neural network rather than $\theta^\top \phi(s, a)$.

Table 1: Neural reward architecture. The network maps discrete state and action indices to a scalar reward through learned embeddings and a feedforward MLP.

Component	Shape	Description
State encoder	$(\lvert\mathcal{S}\rvert, d_{\text{embed}})$ or $f : s \to \mathbb{R}^{d_s}$	Learned embedding or normalized coordinates
Action encoder	$(\lvert\mathcal{A}\rvert, d_{\text{embed}})$ or one-hot	Learned embedding or one-hot encoding
Hidden layers	$[\text{concat}] \to d_h \to d_h$	ReLU activations, typically 2 layers of 64 units
Output	$d_h \to 1$	Scalar reward $R_\psi(s, a)$

The `MCEIRLNeural` estimator in econirl uses a state encoder that maps state indices to normalized coordinates and concatenates them with one-hot action encodings. The `DeepMaxEntIRLEstimator`

in `econirl.contrib` uses learned embedding matrices for both states and actions. Both produce a reward matrix $R_\psi(s,a)$ of shape $(|\mathcal{S}|, |\mathcal{A}|)$ that feeds into the same soft Bellman solver used by linear MCE-IRL.

3 Derivation

3.1 The MCE-IRL Gradient for Neural Rewards

The MCE-IRL objective is the dual of the maximum causal entropy problem. For linear rewards $R = \theta^\top \phi$, the gradient of the dual loss is the feature matching residual (Theorem 2 of the MCE-IRL primer)

$$\nabla_\theta L = \bar{\phi}_\pi(\theta) - \bar{\phi}_{\text{data}}, \tag{2}$$

where $\bar{\phi}_\pi = \sum_s D_\pi(s) \sum_a \pi(a|s)\, \phi(s,a)$ are the expected features under the current policy and $\bar{\phi}_{\text{data}}$ are the empirical feature expectations.

For a neural reward $R_\psi(s,a)$, we need the gradient with respect to the network parameters ψ. The chain rule gives

$$\nabla_\psi L = \sum_{s,a} \big(\mu_\pi(s,a) - \mu_D(s,a)\big)\, \nabla_\psi R_\psi(s,a), \tag{3}$$

where $\mu_D(s,a) = D_{\text{data}}(s)\, \hat{\pi}_{\text{data}}(a|s)$ is the empirical state-action occupancy and $\mu_\pi(s,a) = D_\pi(s)\, \pi(a|s)$ is the model occupancy under the current policy. The term $(\mu_\pi - \mu_D)$ plays the same role as the feature matching residual, but now it weights the gradient of each individual reward value $R_\psi(s,a)$ rather than aggregating through linear features.

3.2 The Surrogate Loss

Computing Equation (3) directly would require differentiating through the soft Bellman fixed point to obtain $\nabla_\psi \mu_\pi$. This is computationally expensive and numerically unstable. Instead, we use the fact that the occupancy mismatch $(\mu_\pi - \mu_D)$ does not depend on ψ for a fixed policy. We treat it as a constant and define the surrogate loss

$$\mathcal{L}(\psi) = -\sum_{s,a} \underbrace{\big[\mu_D(s,a) - \mu_\pi(s,a)\big]}_{\text{stop gradient}} \cdot R_\psi(s,a). \tag{4}$$

The gradient of this surrogate loss with respect to ψ is exactly $\nabla_\psi L$ from Equation (3), because the stop-gradient term is treated as a constant coefficient. This is the standard approach in both Wulfmeier et al. (2016) and the `imitation` library.

3.3 Backward and Forward Passes

The backward pass solves the soft Bellman equations

$$Q(s,a) = R_\psi(s,a) + \beta \sum_{s'} P(s'|s,a)\, V(s'), \tag{5}$$

$$V(s) = \sigma \log \sum_a \exp\big(Q(s,a)/\sigma\big), \tag{6}$$

$$\pi(a|s) = \exp\big((Q(s,a) - V(s))/\sigma\big). \tag{7}$$

This is identical to linear MCE-IRL. The only difference is that $R_\psi(s,a)$ comes from a neural network forward pass rather than a matrix product $\theta^\top \phi$.

The forward pass computes the occupancy measure

$$D_\pi = (I - \beta\, F_\pi^\top)^{-1}\, \rho_0, \tag{8}$$

where $F_\pi[s, s'] = \sum_a \pi(a|s)\, P(s'|s, a)$ and ρ_0 is the initial state distribution. This is also identical to linear MCE-IRL.

The gradient computation is the only step that differs. In the linear case, the gradient is the feature matching residual $\bar{\phi}_\pi - \bar{\phi}_{\text{data}}$, which is a K-dimensional vector. In the neural case, the gradient flows through the network via the surrogate loss in Equation (4), producing updates for all network parameters ψ.

4 Identification and Interpretability

Neural rewards inherit the same identification limitations as linear rewards. Rewards are identified only up to potential-based shaping (Ng et al., 1999). For any function $h : \mathcal{S} \to \mathbb{R}$, the shaped reward $R'(s, a) = R(s, a) + h(s) - \beta \sum_{s'} P(s'|s, a)\, h(s')$ produces the same optimal policy (Kim et al., 2021). The neural network can absorb any such shaping function into its parameters without changing the induced policy.

Post-hoc interpretability is achieved through sieve projection. After training, the neural reward surface $R_\psi(s, a)$ is projected onto a linear feature basis via least squares

$$\hat{\theta} =_\theta \sum_{s,a} \big(R_\psi(s, a) - \theta^\top \phi(s, a)\big)^2. \tag{9}$$

The projection R^2 measures how well the neural reward can be explained by the chosen linear features. A high R^2 (above 0.8) suggests that the true reward is approximately linear and the neural network was unnecessary. A low R^2 (below 0.5) suggests genuine nonlinearities that linear models cannot capture.

Standard errors from the projection regression are pseudo standard errors only. They do not account for estimation error in R_ψ itself, because the neural reward has no closed-form Hessian. Valid inference requires bootstrap resampling of the full training procedure, which is computationally expensive.

5 Algorithm

Algorithm 1: Deep MCE-IRL (Wulfmeier et al., 2016)

Input: Demonstrations $\{(s_i, a_i)\}_{i=1}^N$, transitions $P(s'|s,a)$, discount β, scale σ, learning rate α

`// Step 1: Compute empirical state-action occupancy`
$\mu_D(s,a) = \frac{1}{N}\sum_{i=1}^N \mathbf{1}[s_i = s, a_i = a]$
Compute initial state distribution ρ_0 from first observations

`// Step 2: Initialize neural reward network`
$R_\psi \leftarrow \mathrm{MLP}(\mathrm{state_encoder}, \mathrm{action_encoder}, \mathrm{hidden_layers})$
$\mathrm{optimizer} \leftarrow \mathrm{Adam}(\alpha, \mathrm{gradient_clip} = 1.0)$

`// Step 3: Iterate until convergence or early stopping`
for $k = 0, 1, \ldots, \mathit{max_epochs}$ **do**

`// 3a. Compute reward matrix from neural network`
$R[s,a] \leftarrow R_\psi(s,a) \quad \forall (s,a)$

`// 3b. Backward pass: soft value iteration (no gradient)`
Solve $V = T_\sigma(V; R)$ via hybrid iteration to get V, Q, π

`// 3c. Forward pass: occupancy measure (no gradient)`
$F_\pi[s,s'] = \sum_a \pi(a|s)\, P(s'|s,a)$
$D_\pi = (I - \beta\, F_\pi^\top)^{-1}\, \rho_0$
$\mu_\pi(s,a) = D_\pi(s) \cdot \pi(a|s)$

`// 3d. Surrogate loss with stop-gradient occupancy mismatch`
$\mathcal{L}(\psi) = -\sum_{s,a} [\mu_D(s,a) - \mu_\pi(s,a)]_{\mathrm{sg}} \cdot R_\psi(s,a)$

`// 3e. Backpropagate through neural network`
$g_\psi = \nabla_\psi \mathcal{L}(\psi)$
Clip $\|g_\psi\| \le 1.0$
$\psi \leftarrow \mathrm{Adam_step}(\psi, g_\psi)$

`// 3f. Early stopping`
Track best model by feature matching residual $\|\mu_D - \mu_\pi\|^2$
if *no improvement for 100 epochs* **then break**, restore best model

end

`// Step 4: Sieve projection (optional)`
if *features* ϕ *provided* **then** $\hat{\theta} =_\theta \|R_\psi - \Phi\theta\|^2$, compute R^2

Output: R_ψ (neural reward), V^*, π^*, optional $\hat{\theta}$ and R^2

Implementation notes. The backward pass (Step 3b) uses the same hybrid solver as linear MCE-IRL, starting with successive approximation and switching to Newton-Kantorovich near the fixed point for quadratic convergence. The forward pass (Step 3c) computes the occupancy measure via matrix inversion. Neither the backward nor the forward pass is differentiated through. Gradients flow only through the neural reward network in Step 3e. The Adam optimizer with gradient clipping provides stable training even when the reward surface is poorly conditioned. Early stopping with patience prevents overfitting by restoring the model checkpoint that achieved the lowest feature matching residual.

6 Results

Table 2: 8×8 Objectworld (64 states, $\beta = 0.9$, 10,000 obs). The Objectworld reward is nonlinear in state features, so linear MCE-IRL misspecifies the reward.

Metric	Deep MCE-IRL	Linear MCE-IRL	BC
Policy accuracy (%)	39.1	84.4	78.1
Time (s)	15.1	5.0	0.3
Projection R^2	-0.0026	—	—

We run Deep MCE-IRL (`MCEIRLNeural`), linear MCE-IRL, and behavioral cloning on a 8×8 Objectworld with 64 states, 5 actions, and 2 continuous distance features. Data consists of 10,000 observations from 200 trajectories of length 50 at $\beta = 0.9$.

Linear MCE-IRL achieves 84.4% policy accuracy despite the fact that the Objectworld reward is nonlinear in the continuous distance features. Behavioral cloning reaches 78.1%. The deep variant achieves only 39.1% after 151 epochs, substantially underperforming both baselines.

This result is not surprising for several reasons. The Objectworld has only 64 states, so the reward function has 320 free parameters in the state-action case. The neural network has far more parameters than needed for this small state space, and the MCE-IRL training signal (the occupancy mismatch) provides only a 320-dimensional gradient signal per iteration. With noisy demonstrations (30% random actions), the empirical occupancy is itself noisy, and the neural network overfits to noise in the occupancy rather than learning the true reward structure. The projection R^2 of -0.0026 confirms that the learned neural reward bears no meaningful relationship to the linear features.

Linear MCE-IRL outperforms the deep variant here because the continuous distance features, while not perfectly representing the nonlinear reward, provide a strong enough inductive bias to recover a good policy. The linear model has only 2 parameters and cannot overfit. The deep model has thousands of parameters and insufficient training signal to constrain them.

Deep MCE-IRL is expected to outperform linear MCE-IRL on larger state spaces (32x32 or larger) where the nonlinear reward structure matters more and the training signal is richer. On small grids, the simplicity of the linear model is an advantage, not a limitation.

7 Code

The neural MCE-IRL estimator is implemented in two files. The `MCEIRLNeural` class at `src/econirl/estimators/m` uses a state encoder with one-hot action concatenation and supports both state-only and state-action reward types. The `DeepMaxEntIRLEstimator` class at `src/econirl/contrib/deep_maxent_irl.py` uses learned embedding matrices for states and actions following the `BaseEstimator` interface. Both estimators share the same MCE-IRL training loop with stop-gradient occupancy mismatch.

The companion script `deep_mce_irl_results.py` in this folder generates Table 2. To regenerate:

```
python papers/econirl_package/primers/deep_mce_irl/deep_mce_irl_results.py
```

References

Kuno Kim, Shivam Garg, Kirthevasan Shiragur, and Stefano Ermon. Reward identification in inverse reinforcement learning. *ICML*, 2021.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations. In *ICML*, pages 278–287, 1999.

Markus Wulfmeier, Peter Ondruska, and Ingmar Posner. Maximum entropy deep inverse reinforcement learning. *arXiv:1507.04888*, 2016.

Brian D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.